

PLANO DE TESTES DO SISTEMA

Quick Finance

Sistema	Quick Finance — Controle Financeiro Pessoal
Equipe	Grupo Quick Finance
Integrantes	Membros
Data	Maio de 2025
Versão	V2.0
Disciplina	Pratica

1. Identificação do Projeto

O Quick Finance é um sistema web/mobile de controle financeiro pessoal desenvolvido como projeto acadêmico. A aplicação permite o gerenciamento de receitas, despesas, orçamentos, metas financeiras e relatórios.

2. Introdução

2.1 Objetivo do Plano de Testes

Este documento tem por objetivo estruturar formalmente o processo de verificação e validação do sistema Quick Finance, definindo estratégias, casos de teste, ambiente, cronograma e riscos associados à fase de testes. A finalidade é garantir que o sistema atenda aos requisitos funcionais e não-funcionais especificados, assegurando qualidade e confiabilidade antes da entrega final.

2.2 Descrição do Sistema

O Quick Finance é uma aplicação full-stack composta por:

- Backend em Java com Spring Boot, expondo uma API REST
- Frontend em React com TypeScript e Vite
- Banco de dados PostgreSQL
- Autenticação via JWT (JSON Web Token)

As principais funcionalidades abrangem: cadastro e autenticação de usuários, registro e categorização de transações financeiras, gerenciamento de orçamentos e metas, calculadora financeira e geração de relatórios.

2.3 Importância da Fase de Testes

Após as etapas de levantamento de requisitos, análise, projeto e codificação, a fase de testes é essencial para identificar defeitos antes da entrega, reduzir retrabalho, validar a conformidade com os requisitos e garantir uma experiência de usuário satisfatória. Testes bem planejados aumentam a confiança na entrega e reduzem riscos de falhas em produção.

3. Escopo dos Testes

3.1 O que será testado

- Cadastro e autenticação de usuários (registro, login, JWT)
- CRUD de transações financeiras (receitas e despesas)
- Gerenciamento de categorias (criar, editar, deletar)
- Gerenciamento de orçamentos (Budget) com alertas de limite
- Metas financeiras (Goal) com acompanhamento de progresso
- Geração de relatórios filtrados por período e categoria
- Sistema de notificações
- Calculadora financeira (juros, financiamento)
- Integração frontend ↔ backend via API REST
- Persistência de dados no banco PostgreSQL

3.2 O que não será testado

- Testes de carga e performance (fora do escopo acadêmico)
- Testes de segurança avançados (penetration testing)
- Compatibilidade com navegadores legados (IE, Edge Legado)
- Aplicativos mobile nativos
- Infraestrutura de produção / deploy em nuvem

4. Estratégia de Testes

4.1 Testes Funcionais

Verificam se cada funcionalidade do sistema se comporta conforme os requisitos. São executados para as funcionalidades de cadastro, login, transações, orçamentos, metas e relatórios, cobrindo tanto cenários válidos quanto inválidos.

4.2 Testes de Validação

Asseguram que os campos dos formulários possuem validações adequadas: campos obrigatórios, formatos de e-mail, valores numéricos positivos, datas válidas e limites de caracteres. Testam também a exibição de mensagens de erro apropriadas ao usuário.

4.3 Testes de Interface

Avaliam a consistência visual e a responsividade da interface em diferentes resoluções (desktop e mobile). Verificam navegação entre telas, feedback visual de carregamento, exibição de toasts/alertas e acessibilidade básica.

4.4 Testes de Integração

Validam a comunicação entre o frontend React e o backend Spring Boot via chamadas HTTP/REST, incluindo autenticação JWT nos headers, tratamento de erros HTTP (400, 401, 404, 500) e a correta persistência dos dados no PostgreSQL após cada operação.

4.5 Testes de Usabilidade

Avaliam a facilidade de uso do sistema por meio de observação de usuários realizando tarefas típicas (cadastrar transação, definir meta, visualizar relatório). Identificam pontos de fricção, fluxos confusos e oportunidades de melhoria na experiência do usuário.

5. Ambiente de Testes

Componente	Descrição
Linguagem Backend	Java 17 com Spring Boot 3.x
Linguagem Frontend	TypeScript 5 + React 18 + Vite
Banco de Dados	PostgreSQL 18
Sistema Operacional	Windows 10/11
IDE	IntelliJ IDEA (backend) · VS Code (frontend)
Frameworks	Spring Security, Spring Data JPA, Lombok, Tailwind CSS

Navegadores	Google Chrome 120+, Mozilla Firefox 120+
Ferramentas de Teste	Postman (API), JUnit 5 (unitário), React Testing Library
Porta Backend	http://localhost:8080
Porta Frontend	http://localhost:5173

6. Casos de Teste

Os casos de teste abaixo cobrem cenários válidos, inválidos, tratamento de erros, validação de campos e persistência em banco de dados.

ID	Funcionalidade	Objetivo do Teste	Entrada	Resultado Esperado	Cenário
CT01	Cadastro de Usuário	Validar registro com dados corretos	Nome, e-mail válido, senha forte	Usuário criado, retorno HTTP 201	Válido
CT02	Cadastro de Usuário	Rejeitar e-mail já cadastrado	E-mail já existente no banco	Erro 409 com mensagem adequada	Inválido
CT03	Login / Autenticação	Autenticar usuário com credenciais válidas	E-mail e senha corretos	Token JWT retornado, acesso liberado	Válido
CT04	Login / Autenticação	Rejeitar login com senha incorreta	E-mail válido + senha errada	Erro 401 – credenciais inválidas	Inválido
CT05	Transações	Registrar receita com todos os campos	Tipo INCOME, valor, categoria, data	Transação salva no banco, exibida na lista	Válido
CT06	Transações	Rejeitar transação sem valor	Campos obrigatórios ausentes	Erro de validação 400 exibido ao usuário	Inválido
CT07	Categorias	Criar nova categoria personalizada	Nome, tipo, cor, ícone válidos	Categoria criada e disponível para seleção	Válido
CT08	Orçamento	Criar orçamento com limite mensal	Categoria, valor, período	Orçamento salvo; alerta ao atingir 80%	Válido
CT09	Orçamento	Rejeitar orçamento sem data de término	startDate preenchida, endDate vazia	Erro de validação, campo obrigatório	Inválido
CT10	Metas Financeiras	Criar meta com prazo e valor alvo	Nome, valor, deadline válidos	Meta criada com progresso 0%	Válido
CT11	Metas Financeiras	Atualizar valor atual da meta	Aporte parcial informado	Progresso atualizado no banco e na UI	Válido
CT12	Relatórios	Gerar relatório filtrado por período	Data início e fim válidas	Relatório com totais de receita/despesa	Válido
CT13	Relatórios	Filtrar relatório por categoria	Categoria selecionada	Apenas transações da categoria exibidas	Válido
CT14	Notificações	Exibir notificação de alerta de orçamento	Orçamento > 80% consumido	Notificação criada e listada para o usuário	Válido
CT15	Autenticação JWT	Bloquear acesso sem token	Request sem header Authorization	Resposta 401 – acesso não autorizado	Inválido

7. Cronograma de Testes

Etapa	Responsável	Data Prevista
Elaboração do Plano de Testes	Equipe completa	10/05/2025
Revisão e aprovação do plano	Líder do grupo	11/05/2025
Configuração do ambiente de testes	Equipe de desenvolvimento	12/05/2025
Execução — Testes de Backend (API)	Equipe backend	13/05/2025
Execução — Testes de Frontend	Equipe frontend	14/05/2025
Execução — Testes de Integração	Equipe completa	15/05/2025
Execução — Testes de Usabilidade	Equipe completa	16/05/2025
Registro de defeitos encontrados	Todos os membros	17/05/2025
Correção dos defeitos críticos	Equipe de desenvolvimento	18–19/05/2025
Reteste e validação final	Equipe completa	20/05/2025
Elaboração do relatório de testes	Líder do grupo	21/05/2025

8. Riscos Identificados

Risco	Probabilidade	Impacto	Mitigação
Assinatura incompatível em CategoryService (4 vs 7 parâmetros)	Alta	Crítico	Corrigir o método createCategory para aceitar color, icon e notes antes dos testes
Import ausente de LocalDate em Budget.java	Alta	Crítico	Adicionar import java.time.LocalDate — compilação falhará sem correção
Endpoints de orçamento ausentes no backend (/filter/category, /filter/date, /alerts)	Média	Alto	Implementar os endpoints faltantes ou remover chamadas do frontend
Dependência axios não declarada no package.json do frontend	Alta	Alto	Executar npm install axios antes dos testes de integração
Falha de conexão com banco de dados PostgreSQL	Baixa	Alto	Verificar configurações em application.properties e garantir serviço ativo
Token JWT expirado durante sessão de teste	Média	Médio	Ajustar tempo de expiração para testes ou renovar token entre casos
Campos sem validação no frontend causando dados inválidos	Média	Médio	Testar especificamente CT06, CT09 e validar mensagens de erro na UI
Tipo Notification conflitando com Web API nativa do browser	Baixa	Médio	Renomear o tipo TypeScript para AppNotification e atualizar imports

— *Fim do Documento* —